

VendorBridge — Procurement & Vendor Management ERP

Architecture, Folder Structure & API Contract (MERN Stack) — v2.0

Prepared for Antigravity vibecoding build

August 2026

Contents

1	0. How to Use This Document	4
1.1	Design Reference Files	4
2	1. Tech Stack — MERN, JavaScript Only	5
3	2. Product Architecture	6
4	3. User Journeys	7
4.1	Procurement Officer	7
4.2	Vendor	7
4.3	Manager / Approver	7
4.4	Admin	7
5	4. Data Model (MongoDB / Mongoose)	8
5.1	4.1 Collections & Schemas	8
5.1.1	User	8
5.1.2	Vendor	8
5.1.3	Rfq	8
5.1.4	Quotation	9
5.1.5	Approval	9
5.1.6	PurchaseOrder	9
5.1.7	Invoice	9
5.1.8	ActivityLog	10
5.1.9	Notification	10
5.2	4.2 Relationship Diagram	10
6	5. API Contract (Full, with Payload Samples)	11
6.1	5.1 Conventions	11
6.2	5.2 Auth — /api/v1/auth	12
6.2.1	POST /auth/signup	12
6.2.2	POST /auth/login	12
6.2.3	POST /auth/refresh	13
6.2.4	POST /auth/logout	13
6.2.5	POST /auth/forgot-password	13
6.2.6	POST /auth/reset-password	13
6.2.7	GET /auth/me	13
6.3	5.3 Users (Admin) — /api/v1/users	13
6.3.1	GET /users	14
6.3.2	GET /users/:id -> 200 single User object, or 404 NOT_FOUND.	14

6.3.3	POST /users	14
6.3.4	PATCH /users/:id	14
6.4	5.4 Vendors — /api/v1/vendors	14
6.4.1	GET /vendors	14
6.4.2	GET /vendors/:id	15
6.4.3	POST /vendors	15
6.4.4	PATCH /vendors/:id	15
6.4.5	PATCH /vendors/:id/status	15
6.5	5.5 RFQs — /api/v1/rfqs	16
6.5.1	GET /rfqs	16
6.5.2	GET /rfqs/:id	16
6.5.3	POST /rfqs	17
6.5.4	PATCH /rfqs/:id	17
6.5.5	POST /rfqs/:id/publish	17
6.5.6	POST /rfqs/:id/attachments	17
6.5.7	POST /rfqs/:id/vendors	17
6.5.8	POST /rfqs/:id/cancel	17
6.6	5.6 Quotations — /api/v1/rfqs/:rfqId/quotations and /api/v1/quotations	17
6.6.1	GET /rfqs/:rfqId/quotations	18
6.6.2	GET /quotations/mine?rfqId=64f2a1b1c2d3e4f5a6b7c8d9	18
6.6.3	POST /rfqs/:rfqId/quotations	18
6.6.4	POST /quotations/:id/withdraw	18
6.7	5.7 Comparison & Selection — /api/v1/rfqs/:rfqId/comparison	18
6.7.1	GET /rfqs/:rfqId/comparison	18
6.7.2	POST /rfqs/:rfqId/select-quotation	19
6.8	5.8 Approvals — /api/v1/approvals	19
6.8.1	GET /approvals	20
6.8.2	GET /approvals/:id	20
6.8.3	POST /approvals/:id/decide	20
6.9	5.9 Purchase Orders — /api/v1/purchase-orders	20
6.9.1	GET /purchase-orders	20
6.9.2	GET /purchase-orders/:id	20
6.9.3	POST /purchase-orders	20
6.9.4	GET /purchase-orders/:id/pdf	21
6.10	5.10 Invoices — /api/v1/invoices	21
6.10.1	GET /invoices	21
6.10.2	GET /invoices/:id	21
6.10.3	POST /invoices	21
6.10.4	GET /invoices/:id/pdf	22
6.10.5	POST /invoices/:id/send-email	22
6.10.6	PATCH /invoices/:id/status	22
6.11	5.11 Notifications & Activity — /api/v1/notifications, /api/v1/activity-logs	22
6.11.1	GET /notifications	22
6.11.2	PATCH /notifications/:id/read -> 200, marks read.	22
6.11.3	PATCH /notifications/read-all -> 200.	22
6.11.4	GET /activity-logs	22
6.12	5.12 Reports & Analytics — /api/v1/reports	22
6.12.1	GET /reports/dashboard-summary	23
6.12.2	GET /reports/vendor-performance	23
6.12.3	GET /reports/spend-summary?from=2026-01-01&to=2026-08-20	23
6.12.4	GET /reports/procurement-trends?months=6	23
6.12.5	GET /reports/export?type=spend vendor-performance&format=csv (optional-tier, Section 9)	23
7	6. Backend Architecture (Node.js + Express + MongoDB, Plain JavaScript)	24
7.1	6.1 Folder Structure	24

7.2	6.2 Layered Responsibility	26
7.3	6.3 Cross-Cutting Backend Rules	26
8	7. Frontend Architecture (React + Vite, Plain JavaScript)	27
8.1	7.1 Folder Structure	27
8.2	7.2 State Management Strategy	28
8.3	7.3 API Integration Pattern	28
8.4	7.4 Routing & Role Gating	29
8.5	7.5 Design System	29
9	8. Authentication & Authorization	30
10	9. MVP vs Optional Features	31
10.1	MVP (must work end-to-end for the demo)	31
10.2	Optional / Nice-to-have (only after MVP is solid)	31
11	10. Implementation Phases	32
12	11. Hackathon Demo Strategy (5-10 min)	33
13	12. Risks	34
14	Amendment Log	35

1 0. How to Use This Document

This document is the **single source of truth** for building VendorBridge. It is written so it can be handed directly to an AI vibecoding agent (Antigravity) or a human developer with no additional context needed.

Rules for implementation:

1. Section 5 (**API Contract**) is **FROZEN** once this document is approved. Every backend route must match it exactly — method, path, payload shape, response shape, status codes. Every frontend call must consume it exactly as written.
2. Do not invent new response envelopes, field names, or status codes that aren't defined here. If something is missing, flag it and extend this document first — do not improvise inside a feature branch.
3. Folder structures in Sections E and F are mandatory skeletons. Files may be added inside a module, but the top-level module boundaries should not be reorganized ad hoc.
4. Everything is **plain JavaScript** — no TypeScript, no .ts/.tsx files anywhere in this project.

1.1 Design Reference Files

The team has design references in **.html format** for screen layout/visual styling. At the time of writing this document, no .html design files were attached to this conversation — only the PDF problem statement and an Excalidraw mockup link (<https://app.excalidraw.com/l/65VNwvy7c4X/5ywnm0v3qhl>) were provided.

Action required before/during frontend implementation: upload the .html design files. Once available, Antigravity should: - Extract the color palette, spacing, typography, and component layout from each .html file. - Map each .html file to the corresponding React page listed in Section E (e.g. dashboard.html → src/features/dashboard/pages/DashboardPage.jsx). - Reuse the HTML's markup structure and class names as the starting point for the JSX + Tailwind implementation, rather than redesigning from scratch, so the delivered app visually matches the approved design exactly. - If any .html file conflicts with the UI/UX direction in Section H, the .html file wins — it represents the team's actual approved design.

2 1. Tech Stack — MERN, JavaScript Only

Layer	Technology	Notes
Database	MongoDB (Mongoose ODM)	Document model fits procurement's nested data (RFQ item lists, quotation snapshots) well; Mongoose gives schema validation + middleware without a query language to learn
Backend	Node.js + Express.js	Plain JavaScript (CommonJS or ES Modules — this doc uses ES Modules import/export), no TypeScript
Frontend	React 18 + Vite	Plain JavaScript, .jsx files, no TypeScript
Styling	Tailwind CSS	Utility-first, fast to build a consistent design system
Frontend data/state	React Query (server state) + Zustand (client/UI state)	Query gives built-in loading/error/cache; Zustand for auth/session and small UI flags
Routing	React Router v6	Protected + role-gated routes
Forms	React Hook Form + Yup	Yup instead of Zod (Zod is TS-oriented; Yup is the standard JS-first validation library)
Backend validation	Joi (or Yup — pick one, this doc uses Joi since it's the most common Express-ecosystem choice)	Validates every request body/query/params before it reaches a controller
Auth	JWT (jsonwebtoken) + bcryptjs	Access + refresh token pair
File uploads	Multer (disk storage under /uploads)	RFQ attachments
PDF generation	PDFKit	Invoice / PO PDF generation, streamed to client
Email	Nodemailer	Invoice email sending
Charts	Recharts	Dashboard + reports
HTTP client (frontend)	Axios	Centralized instance with interceptors
Dev/test	Nodemon, dotenv, ESLint + Prettier	Standard JS tooling, no build step for backend

This is a **monorepo** with two independently runnable JavaScript apps: /backend (Express + MongoDB) and /frontend (React + Vite). No shared code between them — the contract in Section 5 is the only thing that couples them.

3 2. Product Architecture

VendorBridge is a role-driven **state machine** wrapped in CRUD screens. The entire product exists to move one procurement request through a fixed sequence of states, where each role owns one or two transitions:

```
RFQ(draft) -> RFQ(published) -> Quotation(submitted, by N vendors)
  -> Quotation(selected) -> Approval(pending)
  -> Approval(approved | rejected)
  -> PurchaseOrder(issued) -> Invoice(generated) -> Invoice(sent | paid)
```

Everything else — dashboard, notifications, activity log, analytics — is a **read/reporting layer** over this one pipeline, not a separate feature to build from scratch. Build the pipeline correctly once; dashboards and reports become derived queries.

System shape: one Express REST API, one MongoDB database, one React SPA.

- The frontend never computes business rules (e.g. “is this quotation still editable?”, “is this the lowest price?”). The backend computes and returns these as flags in the response. The frontend only renders what it’s given.
- Every state transition (RFQ published, quotation submitted, quotation selected, approval decided, PO issued, invoice generated/sent) writes one ActivityLog document and fans out Notification documents, centralized in one backend service (`activityLogger.service.js`) — never duplicated per route.

4 3. User Journeys

4.1 Procurement Officer

1. Logs in -> Dashboard (pending approvals, active RFQs, recent POs/invoices, spend stats).
2. Creates RFQ -> title, item/service list, quantity, deadline, attaches spec file -> assigns 2-4 vendors -> publishes.
3. Watches RFQ detail page fill in as vendors respond ("2 of 4 vendors responded").
4. Opens **Comparison** screen -> sorts by price/delivery/rating -> selects a winning quotation.
5. Selecting a quotation auto-creates an **Approval** request routed to a Manager.
6. Once approved, generates a **Purchase Order** in one click (pre-filled from the approved quotation).
7. Generates the **Invoice** from the PO -> previews -> downloads PDF / prints / emails to the vendor.
8. Dashboard and Reports reflect the completed cycle immediately.

4.2 Vendor

1. Logs in -> Dashboard shows RFQs assigned to them (awaiting response / submitted / won / lost).
2. Opens an RFQ -> reads spec + deadline -> submits a Quotation (unit price, total, delivery timeline, notes).
3. Can edit the quotation until the RFQ deadline **and** until the Officer selects a winner (backend-gated).
4. After selection: sees "Awarded to another vendor," or, if they won, sees the PO once issued.
5. Views/downloads POs and invoices addressed to them.

4.3 Manager / Approver

1. Logs in -> Dashboard's primary widget is **Pending Approvals**.
2. Opens an approval request -> sees RFQ context, the selected quotation, and how it compared against alternatives (not deciding blind).
3. Approves or rejects; remark is mandatory on rejection, optional on approval.
4. Decision instantly unblocks (or halts) PO generation and is written to the audit trail.

4.4 Admin

1. Logs in -> admin-scoped dashboard: user management, vendor directory oversight, org-wide analytics.
2. Manages user accounts (create/deactivate/reassign role).
3. Manages/verifies vendor records (activates newly self-registered vendors, deactivates non-compliant ones).
4. Views the same Reports & Analytics as the Officer, but org-wide, with export.

5 4. Data Model (MongoDB / Mongoose)

Documents use MongoDB's native `_id` (`ObjectId`). Relationships are modeled with `ObjectId` references (`ref`), populated via Mongoose `.populate()` where the API contract requires nested data.

5.1 4.1 Collections & Schemas

5.1.1 User

```
{
  _id: ObjectId,
  name: String,           // required
  email: String,          // required, unique, lowercase
  passwordHash: String,   // required, bcrypt hash
  role: String,           // enum: PROCUREMENT_OFFICER | VENDOR | MANAGER | ADMIN
  vendorId: ObjectId,     // ref: 'Vendor', only set if role === VENDOR
  isActive: Boolean,      // default true
  createdAt: Date,
  updatedAt: Date
}
```

5.1.2 Vendor

```
{
  _id: ObjectId,
  companyName: String,    // required
  gstNumber: String,      // required, unique, 15-char GSTIN format
  category: String,       // e.g. "Electronics", "Office Supplies", "Raw Materials"
  contactName: String,
  contactEmail: String,   // required
  contactPhone: String,   // required
  address: String,
  status: String,         // enum: PENDING | ACTIVE | INACTIVE, default PENDING
  rating: Number,         // 0-5, default 0, manually set for MVP
  createdAt: Date,
  updatedAt: Date
}
```

5.1.3 Rfq

```
{
  _id: ObjectId,
  rfqNumber: String,      // unique, auto-generated e.g. "RFQ-2026-0001"
  title: String,          // required
  description: String,
  itemDetails: [
    { name: String, quantity: Number, unit: String, specification: String }
  ],
  deadline: Date,         // required
  status: String,         // enum: DRAFT | PUBLISHED | CLOSED | AWARDED | CANCELLED
  assignedVendors: [ObjectId], // ref: 'Vendor'
  attachments: [
    { fileName: String, filePath: String, fileSize: Number, uploadedAt: Date }
  ],
  createdBy: ObjectId,    // ref: 'User'
  createdAt: Date,
}
```



```

    updatedAt: Date
}

```

5.1.4 Quotation

```

{
  _id: ObjectId,
  rfqId: ObjectId,           // ref: 'Rfq', required
  vendorId: ObjectId,       // ref: 'Vendor', required
  unitPrice: Number,        // required, > 0
  quantity: Number,        // required, > 0 (copied from RFQ at submit time)
  totalAmount: Number,     // computed: unitPrice * quantity
  deliveryDays: Number,    // required, > 0
  notes: String,
  status: String,          // enum: SUBMITTED | SELECTED | REJECTED | WITHDRAWN
  submittedAt: Date,
  updatedAt: Date
}
// Compound unique index: { rfqId: 1, vendorId: 1 } – one active quotation per vendor per RFQ

```

5.1.5 Approval

```

{
  _id: ObjectId,
  quotationId: ObjectId,    // ref: 'Quotation', unique
  rfqId: ObjectId,         // ref: 'Rfq' (denormalized for fast lookups)
  requestedBy: ObjectId,   // ref: 'User' (the Procurement Officer)
  approverId: ObjectId,    // ref: 'User', nullable until decided
  status: String,          // enum: PENDING | APPROVED | REJECTED
  remarks: String,
  decidedAt: Date,
  createdAt: Date
}

```

5.1.6 PurchaseOrder

```

{
  _id: ObjectId,
  poNumber: String,        // unique, auto-generated e.g. "PO-2026-0001"
  quotationId: ObjectId,   // ref: 'Quotation', unique
  approvalId: ObjectId,   // ref: 'Approval'
  rfqId: ObjectId,        // denormalized
  vendorId: ObjectId,     // denormalized
  issuedBy: ObjectId,     // ref: 'User'
  status: String,         // enum: ISSUED | ACKNOWLEDGED | COMPLETED | CANCELLED
  totalAmount: Number,
  createdAt: Date
}

```

5.1.7 Invoice

```

{
  _id: ObjectId,
  invoiceNumber: String,   // unique, auto-generated e.g. "INV-2026-0001"
  purchaseOrderId: ObjectId, // ref: 'PurchaseOrder', unique
}

```

```

vendorId: ObjectId,           // denormalized
subtotal: Number,
taxRate: Number,             // percentage, e.g. 18 for 18% GST
taxAmount: Number,          // computed
totalAmount: Number,        // computed
status: String,              // enum: GENERATED | SENT | PAID
pdfPath: String,
sentAt: Date,
createdAt: Date
}

```

5.1.8 ActivityLog

```

{
  _id: ObjectId,
  entityType: String,        // enum: RFQ | QUOTATION | APPROVAL | PO | INVOICE | VENDOR | USER
  entityId: ObjectId,
  action: String,            // e.g. "RFQ_PUBLISHED", "QUOTATION_SUBMITTED"
  actorId: ObjectId,         // ref: 'User'
  metadata: Object,          // free-form extra context
  createdAt: Date
}

```

5.1.9 Notification

```

{
  _id: ObjectId,
  userId: ObjectId,          // ref: 'User', recipient
  type: String,
  title: String,
  message: String,
  relatedEntityType: String,
  relatedEntityId: ObjectId,
  isRead: Boolean,           // default false
  createdAt: Date
}

```

5.2 4.2 Relationship Diagram

```

User 1---1 Vendor           (only when role = VENDOR)
User 1---N Rfq               (createdBy)
Rfq N---N Vendor             (assignedVendors array)
Rfq 1---N Quotation
Vendor 1---N Quotation
Quotation 1---1 Approval
Approval 1---1 PurchaseOrder
PurchaseOrder 1---1 Invoice
User 1---N ActivityLog        (actorId)
User 1---N Notification       (userId)

```

The Quotation -> Approval -> PurchaseOrder -> Invoice chain is a strict **1:1 linear chain** on purpose — it matches the mandated workflow exactly and keeps every “show me the full lifecycle of RFQ X” query a simple chain of lookups, not a branching tree.

6 5. API Contract (Full, with Payload Samples)

6.1 5.1 Conventions

Base URL: /api/v1

Success envelope:

```
{
  "success": true,
  "data": { },
  "meta": { }
}
```

meta is present only on paginated list endpoints.

Error envelope:

```
{
  "success": false,
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Human-readable summary",
    "details": [
      { "field": "email", "message": "Invalid email format" }
    ]
  }
}
```

Error codes -> HTTP status:

Code	Status
VALIDATION_ERROR	400
UNAUTHORIZED	401
FORBIDDEN	403
NOT_FOUND	404
CONFLICT	409
INTERNAL_ERROR	500

IDs: MongoDB ObjectId strings (24-char hex) everywhere, e.g. "64f1a2b3c4d5e6f7a8b9c0d1". Human-facing numbers (rfqNumber, poNumber, invoiceNumber) are separate auto-generated strings.

Dates: ISO 8601 UTC on the wire, e.g. "2026-08-20T10:30:00.000Z". Frontend formats for display only.

Pagination: query params page (default 1), limit (default 20, max 100). Response meta:

```
{ "page": 1, "limit": 20, "total": 57, "totalPages": 3 }
```

Filtering: query params named after the field, e.g. ?status=PUBLISHED&category=Electronics.

Sorting: ?sortBy=createdAt&sortOrder=desc — default is createdAt desc everywhere unless stated.

Auth header: Authorization: Bearer <accessToken> required on every protected route.

6.2 5.2 Auth — /api/v1/auth

6.2.1 POST /auth/signup

Access: Public. Creates a VENDOR-role User + a Vendor document (status: "PENDING").

Request body:

```
{
  "companyName": "Shree Balaji Traders",
  "gstNumber": "24ABCDE1234F1Z5",
  "category": "Office Supplies",
  "contactName": "Rakesh Patel",
  "email": "rakesh@shreebalaji.co.in",
  "phone": "+919824012345",
  "password": "SecurePass@123"
}
```

Response 201:

```
{
  "success": true,
  "data": {
    "user": {
      "id": "64f1a2b3c4d5e6f7a8b9c0d1",
      "name": "Rakesh Patel",
      "email": "rakesh@shreebalaji.co.in",
      "role": "VENDOR"
    },
    "vendor": {
      "id": "64f1a2b3c4d5e6f7a8b9c0d2",
      "companyName": "Shree Balaji Traders",
      "status": "PENDING"
    }
  }
}
```

Errors: 400 VALIDATION_ERROR (bad email/GST format, weak password), 409 CONFLICT (email or GST already exists).

6.2.2 POST /auth/login

Access: Public.

Request body:

```
{ "email": "officer@vendorbridge.com", "password": "SecurePass@123" }
```

Response 200:

```
{
  "success": true,
  "data": {
    "accessToken": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
    "user": {
      "id": "64f1a2b3c4d5e6f7a8b9c0d1",
      "name": "Anita Sharma",
      "email": "officer@vendorbridge.com",
      "role": "PROCUREMENT_OFFICER",
    }
  }
}
```

```
    "vendorId": null
  }
}
```

A refresh token is also set as an **httpOnly cookie** (refreshToken, 7-day expiry) — not present in the JSON body.

Errors: 400 VALIDATION_ERROR, 401 UNAUTHORIZED (bad credentials or isActive: false).

6.2.3 POST /auth/refresh

Access: Public, requires valid refreshToken cookie. **Request body:** none. **Response 200:** new { accessToken }, rotates the refresh cookie. **Errors:** 401 UNAUTHORIZED (missing/expired/invalid cookie).

6.2.4 POST /auth/logout

Access: Authenticated, any role. Clears the refresh cookie. **Response 200:** { "success": true, "data": null }

6.2.5 POST /auth/forgot-password

Access: Public. **Request body:** { "email": "vendor@example.com" } **Response 200** (always, to avoid user enumeration):

```
{ "success": true, "data": { "message": "If that email exists, a reset link has been sent." } }
```

6.2.6 POST /auth/reset-password

Access: Public. **Request body:**

```
{ "token": "a1b2c3d4-reset-token", "newPassword": "NewSecurePass@456" }
```

Response 200: { "success": true, "data": { "message": "Password reset successfully." } } **Errors:** 400 VALIDATION_ERROR (weak password), 401 UNAUTHORIZED (bad/expired token).

6.2.7 GET /auth/me

Access: Authenticated, any role. **Response 200:** current user's profile object (same shape as the login response's user).

6.3 5.3 Users (Admin) — /api/v1/users

Access: ADMIN only.

6.3.1 GET /users

Query: ?role=&isActive=&page=&limit=&sortBy=&sortOrder= **Response 200:**

```
{
  "success": true,
  "data": [
    { "id": "64f1...", "name": "Anita Sharma", "email": "officer@vendorbridge.com", "role": "PRO"
  ],
  "meta": { "page": 1, "limit": 20, "total": 12, "totalPages": 1 }
}
```

6.3.2 GET /users/:id -> 200 single User object, or 404 NOT_FOUND.

6.3.3 POST /users

Creates a staff account (Officer / Manager / Admin). **Request body:**

```
{ "name": "Vikram Rao", "email": "vikram.rao@vendorbridge.com", "password": "TempPass@789", "role": "PRO" }
```

Response 201: created User object (without passwordHash).

6.3.4 PATCH /users/:id

Request body (any subset):

```
{ "isActive": false }
```

Response 200: updated User. 404 NOT_FOUND / 409 CONFLICT as applicable.

6.4 5.4 Vendors — /api/v1/vendors

Access: Read -> Officer, Manager, Admin (all vendors), Vendor (own record only). Write -> Admin (status), Admin/Officer (create on behalf), Vendor (own profile fields).

6.4.1 GET /vendors

Query: ?status=&category=&search=&page=&limit=&sortBy=&sortOrder= (search matches companyName or contactEmail) **Response 200:**

```
{
  "success": true,
  "data": [
    {
      "id": "64f1a2b3c4d5e6f7a8b9c0d2",
      "companyName": "Shree Balaji Traders",
      "gstNumber": "24ABCDE1234F1Z5",
      "category": "Office Supplies",
      "status": "ACTIVE",
      "rating": 4.2,
      "contactEmail": "rakesh@shreebalaji.co.in",
      "contactPhone": "+919824012345"
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 34, "totalPages": 2 }
}
```

6.4.2 GET /vendors/:id

Response 200: full Vendor object plus computed fields:

```
{
  "success": true,
  "data": {
    "id": "64f1a2b3c4d5e6f7a8b9c0d2",
    "companyName": "Shree Balaji Traders",
    "gstNumber": "24ABCDE1234F1Z5",
    "category": "Office Supplies",
    "contactName": "Rakesh Patel",
    "contactEmail": "rakesh@shreebalaji.co.in",
    "contactPhone": "+919824012345",
    "address": "B-12, GIDC Industrial Estate, Surat, Gujarat",
    "status": "ACTIVE",
    "rating": 4.2,
    "totalRfqsInvited": 18,
    "totalQuotationsWon": 6,
    "createdAt": "2026-01-15T08:00:00.000Z"
  }
}
```

404 NOT_FOUND if missing.

6.4.3 POST /vendors

Access: Admin, Officer. **Request body:**

```
{
  "companyName": "Nova Electronics Pvt Ltd",
  "gstNumber": "27PQRSX5678L1Z9",
  "category": "Electronics",
  "contactName": "Sunita Deshmukh",
  "contactEmail": "sunita@novaelectronics.in",
  "contactPhone": "+919876543210",
  "address": "Plot 45, MIDC, Pune, Maharashtra"
}
```

Response 201: created Vendor (status: "PENDING" by default when created by Officer, "ACTIVE" when created by Admin). **Errors:** 400 VALIDATION_ERROR (invalid GSTIN format — 15 chars, standard GSTIN regex), 409 CONFLICT (duplicate GST).

6.4.4 PATCH /vendors/:id

Access: Admin (any field), Vendor (own contactName/contactEmail/contactPhone/address only — enforced server-side). **Request body (subset):**

```
{ "contactPhone": "+919824099999" }
```

Response 200: updated Vendor. 403 FORBIDDEN if a Vendor edits someone else's record.

6.4.5 PATCH /vendors/:id/status

Access: Admin only. **Request body:**

```
{ "status": "ACTIVE" }
```

Response 200: updated Vendor. Triggers a Notification to the vendor's user account.

6.5 5.5 RFQs — /api/v1/rfqs

Access: Create/manage -> Officer, Admin. Read -> Officer/Admin (all), Manager (all, read-only), Vendor (only RFQs where their vendorId is in assignedVendors — backend auto-scopes regardless of filters passed).

6.5.1 GET /rfqs

Query: ?status=&createdBy=&page=&limit=&sortBy=&sortOrder= **Response 200:**

```
{
  "success": true,
  "data": [
    {
      "id": "64f2a1b1c2d3e4f5a6b7c8d9",
      "rfqNumber": "RFQ-2026-0014",
      "title": "Procurement of Office Laptops (25 units)",
      "status": "PUBLISHED",
      "deadline": "2026-09-05T18:30:00.000Z",
      "assignedVendorCount": 3,
      "quotationCount": 2,
      "createdAt": "2026-08-18T07:12:00.000Z"
    }
  ],
  "meta": { "page": 1, "limit": 20, "total": 41, "totalPages": 3 }
}
```

6.5.2 GET /rfqs/:id

Response 200:

```
{
  "success": true,
  "data": {
    "id": "64f2a1b1c2d3e4f5a6b7c8d9",
    "rfqNumber": "RFQ-2026-0014",
    "title": "Procurement of Office Laptops (25 units)",
    "description": "Business-grade laptops for the new Pune office setup.",
    "itemDetails": [
      { "name": "Business Laptop, 16GB RAM, 512GB SSD", "quantity": 25, "unit": "unit", "specific": "" }
    ],
    "deadline": "2026-09-05T18:30:00.000Z",
    "status": "PUBLISHED",
    "attachments": [
      { "fileName": "laptop-spec-sheet.pdf", "filePath": "/uploads/rfq/64f2.../laptop-spec-sheet.pdf" }
    ],
    "assignedVendors": [
      { "id": "64f1a2b3c4d5e6f7a8b9c0d2", "companyName": "Shree Balaji Traders", "responded": true },
      { "id": "64f1a2b3c4d5e6f7a8b9c0d5", "companyName": "Nova Electronics Pvt Ltd", "responded": false }
    ],
    "createdBy": { "id": "64f1a2b3c4d5e6f7a8b9c0d1", "name": "Anita Sharma" },
    "createdAt": "2026-08-18T07:12:00.000Z"
  }
}
```


403 FORBIDDEN if a Vendor requests an RFQ they aren't assigned to. 404 NOT_FOUND.

6.5.3 POST /rfqs

Access: Officer, Admin. **Request body:**

```
{
  "title": "Procurement of Office Laptops (25 units)",
  "description": "Business-grade laptops for the new Pune office setup.",
  "itemDetails": [
    { "name": "Business Laptop, 16GB RAM, 512GB SSD", "quantity": 25, "unit": "unit", "specificat
  ],
  "deadline": "2026-09-05T18:30:00.000Z",
  "vendorIds": ["64f1a2b3c4d5e6f7a8b9c0d2", "64f1a2b3c4d5e6f7a8b9c0d5"],
  "publish": true
}
```

publish: true creates directly as PUBLISHED (and notifies vendors); publish: false (or omitted) creates as DRAFT. **Response 201:** created Rfq object including generated rfqNumber. **Errors:** 400 VALIDATION_ERROR (empty itemDetails, deadline in the past, fewer than 1 vendorIds).

6.5.4 PATCH /rfqs/:id

Access: Officer (own RFQs only), Admin. Editable only while status === "DRAFT". **Request body (subset):**

```
{ "title": "Procurement of Office Laptops (30 units)", "deadline": "2026-09-10T18:30:00.000Z" }
```

Response 200: updated Rfq. 409 CONFLICT if already published.

6.5.5 POST /rfqs/:id/publish

Access: Officer, Admin. Transitions DRAFT -> PUBLISHED, fans out Notification to assigned vendors. **Request body:** none. **Response 200:** updated Rfq. 409 CONFLICT if not currently DRAFT.

6.5.6 POST /rfqs/:id/attachments

Access: Officer (own), Admin. multipart/form-data, field name file. **Response 201:**

```
{ "success": true, "data": { "fileName": "laptop-spec-sheet.pdf", "filePath": "/uploads/rfq/64f2..." }
```

6.5.7 POST /rfqs/:id/vendors

Adds vendor assignment(s) post-creation. **Request body:** { "vendorIds": ["64f1a2b3c4d5e6f7a8b9c0d9"] } **Response 200:** updated Rfq.

6.5.8 POST /rfqs/:id/cancel

Access: Officer (own), Admin. **Request body:** { "reason": "Requirement withdrawn by finance team." } **Response 200:** updated Rfq (status: "CANCELLED"), notifies assigned vendors.

6.6 5.6 Quotations — /api/v1/rfqs/:rfqId/quotations and /api/v1/quotations

Access: Submit/edit -> Vendor (only for RFQs they're assigned to, only while RPUBLISHED, before deadline, before a winner is selected). Read all-for-RFQ -> Officer/Admin/Manager.

6.6.1 GET /rfqs/:rfqId/quotations

Access: Officer, Admin, Manager. (Vendors get 403 FORBIDDEN — they use /quotations/mine below.) **Response 200:**

```
{
  "success": true,
  "data": [
    {
      "id": "64f3b1c1d2e3f4a5b6c7d8e0",
      "vendor": { "id": "64f1a2b3c4d5e6f7a8b9c0d2", "companyName": "Shree Balaji Traders", "rating": 4.5 },
      "unitPrice": 54500,
      "quantity": 25,
      "totalAmount": 1362500,
      "deliveryDays": 12,
      "notes": "Includes 3-year onsite warranty.",
      "status": "SUBMITTED",
      "submittedAt": "2026-08-19T11:00:00.000Z"
    }
  ]
}
```

6.6.2 GET /quotations/mine?rfqId=64f2a1b1c2d3e4f5a6b7c8d9

Access: Vendor. Returns the caller's own quotation for that RFQ, or data: null (valid empty state, not a 404) if not yet submitted.

6.6.3 POST /rfqs/:rfqId/quotations

Access: Vendor (must be in the RFQ's assignedVendors). Upserts — creates on first call, updates on subsequent calls while still editable. **Request body:**

```
{ "unitPrice": 54500, "deliveryDays": 12, "notes": "Includes 3-year onsite warranty." }
```

(quantity is copied server-side from the RFQ's itemDetails, not supplied by the client.) **Response 201 (create) / 200 (update):** the Quotation object. **Errors:** 403 FORBIDDEN (not assigned), 409 CONFLICT (deadline passed, RFQ not PUBLISHED, or a winner already selected), 400 VALIDATION_ERROR (unitPrice <= 0, deliveryDays <= 0).

6.6.4 POST /quotations/:id/withdraw

Access: Vendor (own quotation only). **Request body:** none. **Response 200:** updated Quotation (status: "WITHDRAWN"). 409 CONFLICT if the RFQ is already AWARDED.

6.7 5.7 Comparison & Selection — /api/v1/rfqs/:rfqId/comparison

Access: Read -> Officer, Admin, Manager. Select (write) -> Officer, Admin.

6.7.1 GET /rfqs/:rfqId/comparison

Response 200:

```
{
  "success": true,
  "data": {
    "rfq": { "id": "64f2a1b1c2d3e4f5a6b7c8d9", "rfqNumber": "RFQ-2026-0014", "title": "Procurement of IT Services" }
  }
}
```

```

"quotations": [
  {
    "id": "64f3b1c1d2e3f4a5b6c7d8e0",
    "vendor": { "id": "64f1a2b3c4d5e6f7a8b9c0d2", "companyName": "Shree Balaji Traders", "rat
    "unitPrice": 54500,
    "totalAmount": 1362500,
    "deliveryDays": 12,
    "notes": "Includes 3-year onsite warranty.",
    "isLowestPrice": true,
    "isFastestDelivery": false
  },
  {
    "id": "64f3b1c1d2e3f4a5b6c7d8e5",
    "vendor": { "id": "64f1a2b3c4d5e6f7a8b9c0d5", "companyName": "Nova Electronics Pvt Ltd",
    "unitPrice": 56200,
    "totalAmount": 1405000,
    "deliveryDays": 8,
    "notes": "Ships from Pune warehouse, faster turnaround.",
    "isLowestPrice": false,
    "isFastestDelivery": true
  }
]
}
}

```

All comparison flags (isLowestPrice, isFastestDelivery) are computed server-side.

6.7.2 POST /rfqs/:rfqId/select-quotation

Access: Officer, Admin. The pivot endpoint of the whole workflow. **Request body:** { "quotationId": "64f3b1c1d2e3f4a5b6c7d8e0" } Sets the chosen quotation status: "SELECTED", all sibling quotations status: "REJECTED", the RFQ status: "AWARDED", and **auto-creates the Approval document** (status: "PENDING"). **Response 200:** the created Approval object:

```

{
  "success": true,
  "data": {
    "id": "64f4c1d1e2f3a4b5c6d7e8f1",
    "quotationId": "64f3b1c1d2e3f4a5b6c7d8e0",
    "rfqId": "64f2a1b1c2d3e4f5a6b7c8d9",
    "requestedBy": "64f1a2b3c4d5e6f7a8b9c0d1",
    "status": "PENDING",
    "createdAt": "2026-08-20T09:00:00.000Z"
  }
}

```

Errors: 409 CONFLICT (RFQ not in a selectable state, or quotationId doesn't belong to this RFQ).

6.8 5.8 Approvals — /api/v1/approvals

Access: Read queue/decide -> Manager, Admin. Read own request status -> Officer (for RFQs they created).

6.8.1 GET /approvals

Query: ?status=&page=&limit= **Response 200:** list of Approval with nested quotation + rfq summary, meta pagination.

6.8.2 GET /approvals/:id

Response 200:

```
{  
  "success": true,  
  "data": {  
    "id": "64f4c1d1e2f3a4b5c6d7e8f1",  
    "status": "PENDING",  
    "rfq": { "id": "64f2a1b1c2d3e4f5a6b7c8d9", "rfqNumber": "RFQ-2026-0014", "title": "Procurement  
quotation": { "id": "64f3b1c1d2e3f4a5b6c7d8e0", "vendor": { "companyName": "Shree Balaji Tra  
alternativeQuotations": [  
  { "vendor": { "companyName": "Nova Electronics Pvt Ltd" }, "unitPrice": 56200, "totalAmount  
],  
  "requestedBy": { "id": "64f1a2b3c4d5e6f7a8b9c0d1", "name": "Anita Sharma" },  
  "createdAt": "2026-08-20T09:00:00.000Z"  
}  
}
```

404 NOT_FOUND.

6.8.3 POST /approvals/:id/decide

Access: Manager, Admin. **Request body:**

```
{ "decision": "APPROVED", "remarks": "Pricing and warranty terms are acceptable." }
```

remarks is **required** if decision == "REJECTED". **Response 200:** updated Approval. On APPROVED, the quotation stays SELECTED and becomes eligible for PO generation. On REJECTED, the RFQ reverts to PUBLISHED so the Officer can select a different quotation. **Errors:** 409 CONFLICT (already decided), 400 VALIDATION ERROR (missing remarks on rejection).

6.9 5.9 Purchase Orders — /api/v1/purchase-orders

Access: Create -> Officer, Admin (only from an APPROVED approval). Read -> Officer/Admin/Manager (all), Vendor (own only).

6.9.1 GET /purchase-orders

Query: ?status=&vendorId=&page=&limit= (Vendor role is auto-scoped to their own POs regardless of vendorId passed). **Response 200:** paginated list.

6.9.2 GET /purchase-orders/:id

Response 200: full PO with nested quotation/rfq/vendor summary. 403 FORBIDDEN if a Vendor requests a PO that isn't theirs.

6.9.3 POST /purchase-orders

Access: Officer, Admin. **Request body:**

```
{ "approvalId": "64f4c1d1e2f3a4b5c6d7e8f1" }
```

Response 201:

```
{
  "success": true,
  "data": {
    "id": "64f5d1e1f2a3b4c5d6e7f8a2",
    "poNumber": "PO-2026-0009",
    "quotationId": "64f3b1c1d2e3f4a5b6c7d8e0",
    "approvalId": "64f4c1d1e2f3a4b5c6d7e8f1",
    "vendorId": "64f1a2b3c4d5e6f7a8b9c0d2",
    "status": "ISSUED",
    "totalAmount": 1362500,
    "createdAt": "2026-08-20T09:15:00.000Z"
  }
}
```

Errors: 409 CONFLICT (approval.status !== "APPROVED", or a PO already exists for it).

6.9.4 GET /purchase-orders/:id/pdf

Streams the PO as a PDF. 200, Content-Type: application/pdf.

6.10 5.10 Invoices — /api/v1/invoices

Access: Generate -> Officer, Admin. Read -> Officer/Admin/Manager (all), Vendor (own only).

6.10.1 GET /invoices

Query: ?status=&vendorId=&page=&limit= **Response 200:** paginated list.

6.10.2 GET /invoices/:id

Response 200: full invoice with tax breakdown.

6.10.3 POST /invoices

Access: Officer, Admin. **Request body:**

```
{ "purchaseOrderId": "64f5d1e1f2a3b4c5d6e7f8a2", "taxRate": 18 }
```

Response 201:

```
{
  "success": true,
  "data": {
    "id": "64f6e1f1a2b3c4d5e6f7a8b3",
    "invoiceNumber": "INV-2026-0009",
    "purchaseOrderId": "64f5d1e1f2a3b4c5d6e7f8a2",
    "vendorId": "64f1a2b3c4d5e6f7a8b9c0d2",
    "subtotal": 1362500,
    "taxRate": 18,
    "taxAmount": 245250,
    "totalAmount": 1607750,
    "status": "GENERATED",
    "createdAt": "2026-08-20T09:20:00.000Z"
  }
}
```

```
}  
}
```

Errors: 409 CONFLICT (an invoice already exists for this PO).

6.10.4 GET /invoices/:id/pdf

Streams the invoice PDF. 200, Content-Type: application/pdf.

6.10.5 POST /invoices/:id/send-email

Access: Officer, Admin. **Request body (optional override):**

```
{ "toEmail": "rakesh@shreebalaji.co.in" }
```

Defaults to the vendor's contactEmail if omitted. Sends the PDF as an attachment via email.service.js, sets status: "SENT", sentAt. **Response 200:** updated Invoice. **Errors:** 502 (mapped to INTERNAL_ERROR) if SMTP fails — the PDF remains downloadable as a fallback.

6.10.6 PATCH /invoices/:id/status

Access: Officer, Admin. **Request body:** { "status": "PAID" } **Response 200:** updated Invoice.

6.11 5.11 Notifications & Activity — /api/v1/notifications, /api/v1/activity-logs

6.11.1 GET /notifications

Access: any authenticated role, auto-scoped to req.user.id. Query: ?isRead=&page=&limit=.

6.11.2 PATCH /notifications/:id/read -> 200, marks read.

6.11.3 PATCH /notifications/read-all -> 200.

6.11.4 GET /activity-logs

Access: Officer, Manager, Admin. Query: ?entityType=&entityId=&page=&limit= (always sorted createdAt desc). **Response 200:**

```
{  
  "success": true,  
  "data": [  
    { "id": "64f7...", "entityType": "RFQ", "entityId": "64f2a1b1c2d3e4f5a6b7c8d9", "action": "RFQ_CREATED", "createdAt": "2023-07-15T10:30:00Z", "updatedAt": "2023-07-15T10:30:00Z" },  
    { "id": "64f8...", "entityType": "QUOTATION", "entityId": "64f3b1c1d2e3f4a5b6c7d8e0", "action": "QUOTATION_CREATED", "createdAt": "2023-07-15T11:00:00Z", "updatedAt": "2023-07-15T11:00:00Z" },  
  ],  
  "meta": { "page": 1, "limit": 20, "total": 9, "totalPages": 1 }  
}
```

6.12 5.12 Reports & Analytics — /api/v1/reports

Access: Officer, Admin, Manager (read). No Vendor access.

6.12.1 GET /reports/dashboard-summary

Role-aware — returns the widget set relevant to req.user.role. **Response 200 (Procurement Officer example):**

```
{
  "success": true,
  "data": {
    "pendingApprovalsCount": 3,
    "activeRfqsCount": 7,
    "recentPurchaseOrders": [ { "poNumber": "PO-2026-0009", "vendor": "Shree Balaji Traders", "to
    "recentInvoices": [ { "invoiceNumber": "INV-2026-0009", "status": "SENT", "totalAmount": 1607
    "spendToDate": 8450000
  }
}
```

6.12.2 GET /reports/vendor-performance

Response 200:

```
{
  "success": true,
  "data": [
    { "vendorId": "64f1a2b3c4d5e6f7a8b9c0d2", "companyName": "Shree Balaji Traders", "quotationsS
  ]
}
```

6.12.3 GET /reports/spend-summary?from=2026-01-01&to=2026-08-20

Response 200: { "totalSpend": ..., "spendByCategory": [...], "spendByVendor": [...] }

6.12.4 GET /reports/procurement-trends?months=6

Response 200: monthly series of RFQs created, POs issued, total spend — feeds the trend chart.

6.12.5 GET /reports/export?type=spend|vendor-performance&format=csv *(optional-tier, Section 9)*

Streams CSV, 200, Content-Type: text/csv.

7 6. Backend Architecture (Node.js + Express + MongoDB, Plain JavaScript)

7.1 6.1 Folder Structure

```
backend/  
  src/  
    server.js          # bootstraps http server, connects Mongo, starts listener  
    app.js             # express app: middleware wiring, route mounting  
    config/  
      env.js           # loads & validates .env vars (throws on startup if missing)  
      db.js            # mongoose.connect() wrapper  
    middleware/  
      auth.middleware.js # verifies JWT, attaches req.user  
      role.middleware.js # requireRole(['MANAGER', 'ADMIN']) guard factory  
      validate.middleware.js # runs a Joi schema against req.body / req.query / req.params  
      error.middleware.js # central error handler -> standard error envelope  
      rateLimiter.middleware.js # express-rate-limit, applied to auth + write-  
heavy routes  
  upload.middleware.js # multer config for RFQ attachments  
  modules/  
    auth/  
      auth.routes.js  
      auth.controller.js  
      auth.service.js # login/signup/forgot-password logic, token issuing  
      auth.schema.js  # Joi request schemas  
    users/  
      users.routes.js  
      users.controller.js  
      users.service.js  
      users.schema.js  
    vendors/  
      vendors.routes.js  
      vendors.controller.js  
      vendors.service.js  
      vendors.schema.js  
      vendor.model.js # Mongoose model  
    rfq/  
      rfq.routes.js  
      rfq.controller.js  
      rfq.service.js  
      rfq.schema.js  
      rfq.model.js  
    quotations/  
      quotations.routes.js  
      quotations.controller.js  
      quotations.service.js  
      quotations.schema.js  
      quotation.model.js  
    comparison/  
      comparison.routes.js  
      comparison.controller.js  
      comparison.service.js # read-only aggregation + selection transition  
  approvals/  
    approvals.routes.js
```



```

    approvals.controller.js
    approvals.service.js
    approvals.schema.js
    approval.model.js
purchaseOrders/
    purchaseOrders.routes.js
    purchaseOrders.controller.js
    purchaseOrders.service.js
    purchaseOrder.model.js
invoices/
    invoices.routes.js
    invoices.controller.js
    invoices.service.js
    invoices.schema.js
    invoice.model.js
notifications/
    notifications.routes.js
    notifications.controller.js
    notifications.service.js
    notification.model.js
activityLogs/
    activityLogs.routes.js
    activityLogs.controller.js
    activityLog.model.js
reports/
    reports.routes.js
    reports.controller.js
    reports.service.js          # Mongo aggregation pipelines
user/
    user.model.js              # Mongoose model kept in users/ module
shared/
    services/
    activityLogger.service.js   # single writer for ActivityLog + Notification fan-out
    pdf.service.js             # invoice/PO PDF generation (PDFKit)
    email.service.js           # nodemailer wrapper
    numberGenerator.service.js  # RFQ/PO/Invoice sequential number generation
    utils/
        apiResponse.js         # success()/error() envelope helpers
    asyncHandler.js            # wraps async controllers to forward errors to error.middleware
    pagination.js              # parses page/limit/sortBy/sortOrder from req.query
seed/
    seed.js                    # populates realistic demo data (Section 9)
    data/ (vendors.json, users.json, rfqs.json)
uploads/
    rfq/                       # multer-saved attachments, served statically
    invoices/                   # generated invoice PDFs
.env.example
package.json

```

Each modules/<name> follows the same 4-file pattern (routes -> controller -> service -> schema, plus a model.js where the module owns a collection). Any two developers can work on separate modules without touching shared files — important when splitting hackathon work across a team.

7.2 6.2 Layered Responsibility

- **Routes (*.routes.js):** only wiring — path, middleware chain (auth, role, validate), controller method reference. No logic.
- **Controllers (*.controller.js):** parse req, call the service, shape the response via apiResponse.js. No business logic, no direct Mongoose calls.
- **Services (*.service.js):** all business logic and Mongoose queries live here. This is where state-transition rules are enforced (e.g. “a quotation can only be edited while the parent RFQ is PUBLISHED, before its deadline, and before a winner is selected”).
- **Schemas (*.schema.js):** Joi definitions for request validation, consumed by validate.middleware.js. These mirror Section 5 exactly — if a payload sample in this document changes, the schema changes too, not the other way around.
- **Models (*.model.js):** Mongoose schema + model definitions matching Section 4 exactly, including indexes (e.g. compound unique index on Quotation { rfqId, vendorId }).

7.3 6.3 Cross-Cutting Backend Rules

- Every service call that changes a tracked entity’s status ends with a call to activityLogger.service.js, which (a) writes an ActivityLog document and (b) creates Notification document(s) for the relevant recipients (e.g. quotation submitted -> notify the RFQ’s createdBy; approval decided -> notify the Procurement Officer). This keeps notification logic out of every individual module.
- Sequential human-facing numbers (RFQ-2026-0001, PO-2026-0001, INV-2026-0001) are generated inside numberGenerator.service.js using a dedicated Counter collection with an atomic findOneAndUpdate({ \$inc: { seq: 1 } }, { upsert: true }) to avoid race conditions under concurrent creation — this is the MongoDB-native equivalent of a DB-transaction-guarded sequence.
- All monetary values are stored as plain Number in MongoDB but always computed/rounded to 2 decimal places server-side before being written or returned, to keep tax/total calculations consistent.
- app.js mounts, in order: helmet() (security headers) -> cors() (locked to frontend origin) -> express.json() -> route modules -> error.middleware.js (must be mounted last).

8 7. Frontend Architecture (React + Vite, Plain JavaScript)

8.1 7.1 Folder Structure

```
frontend/  
  src/  
    main.jsx          # ReactDOM root render  
    App.jsx           # top-level app shell  
  app/  
    router.jsx        # route table, ProtectedRoute / RoleRoute wrappers  
    providers.jsx     # QueryClientProvider, AuthProvider, ToastProvider  
  features/  
    auth/  
      pages/  
        LoginPage.jsx  
        SignupPage.jsx  
        ForgotPasswordPage.jsx  
    api.js            # login(), signup(), forgotPassword() – axios calls  
    useAuthStore.js   # Zustand store: user, accessToken, role  
    authSchema.js     # Yup validation schemas for the forms  
  dashboard/  
    pages/DashboardPage.jsx  
    components/  
      PendingApprovalsCard.jsx  
      ActiveRfqsCard.jsx  
      RecentPOsCard.jsx  
      RecentInvoicesCard.jsx  
      StatsCard.jsx  
      QuickActions.jsx  
    api.js  
    hooks.js          # useDashboardSummary() React Query hook  
  vendors/  
    pages/ (VendorListPage.jsx, VendorDetailPage.jsx, VendorFormPage.jsx)  
    components/ (VendorTable.jsx, VendorStatusBadge.jsx, VendorFilterBar.jsx)  
    api.js / hooks.js  
  rfq/  
    pages/ (RfqListPage.jsx, RfqCreatePage.jsx, RfqDetailPage.jsx)  
  components/ (RfqForm.jsx, RfqStatusBadge.jsx, VendorAssignPicker.jsx, AttachmentUploader.jsx)  
    api.js / hooks.js  
  quotations/  
    pages/ (QuotationSubmitPage.jsx, VendorQuotationsPage.jsx)  
    components/ (QuotationForm.jsx, QuotationCard.jsx)  
    api.js / hooks.js  
  comparison/  
    pages/ComparisonPage.jsx  
  components/ (ComparisonTable.jsx, LowestPriceHighlight.jsx, RatingIndicator.jsx)  
    api.js / hooks.js  
  approvals/  
    pages/ (ApprovalQueuePage.jsx, ApprovalDetailPage.jsx)  
    components/ (ApprovalTimeline.jsx, DecisionForm.jsx)  
    api.js / hooks.js  
  procurementDocs/   # PO + Invoice share a lot of UI  
    pages/ (PurchaseOrderPage.jsx, InvoicePage.jsx)  
    components/ (DocumentPreview.jsx, TaxBreakdownTable.jsx, EmailInvoiceModal.jsx)  
    api.js / hooks.js  
  notifications/
```

```

    components/ (NotificationBell.jsx, NotificationList.jsx, ActivityTimeline.jsx)
    api.js / hooks.js
  reports/
    pages/ReportsPage.jsx
  components/ (SpendChart.jsx, VendorPerformanceChart.jsx, TrendChart.jsx, ExportButton.jsx)
    api.js / hooks.js
  admin/
    pages/ (UserManagementPage.jsx, VendorApprovalPage.jsx)
    api.js / hooks.js
  components/ui/                                # design-system primitives
    Button.jsx
    Input.jsx
    Select.jsx
    Table.jsx
    Card.jsx
    Badge.jsx
    Modal.jsx
    Toast.jsx
    Skeleton.jsx
    EmptyState.jsx
    ErrorState.jsx
  lib/
    apiClient.js                                # single axios instance: base URL, auth header injection,
                                                # 401 refresh-and-
retry interceptor, error envelope unwrapping
    queryClient.js                              # React Query client config
    formatters.js                              # date, currency (INR), status-label formatting
    permissions.js                             # role -> allowed-actions map (UI-
only gating, mirrors backend)
    designReference/                            # <-- team's approved .html design files go here once provided;
                                                #   used as the visual/markup source of truth per Section 0
    assets/
      logo.svg
    index.css                                  # Tailwind directives + design tokens (CSS variables)
  vite.config.js
  tailwind.config.js
  .env.example
  package.json

```

8.2 7.2 State Management Strategy

- **Server state** (anything from the API — RFQs, quotations, vendors, POs...) -> **React Query only**. Every list/detail view is a useQuery; every mutation is a useMutation that invalidates the relevant query keys. This is what gives Loading / Success / Empty / Error handling for free across the app (Section 10 of the brief).
- **Client/UI state** (current modal open, selected comparison rows, sidebar collapsed, auth user/token) -> **Zustand**, kept intentionally small.
- **Forms** -> **React Hook Form** + **Yup** resolver; Yup schemas colocated with each feature's api.js (e.g. rfqSchema.js next to rfq/api.js) so payload shape and validation live side-by-side, and match Section 5's request-body samples exactly.

8.3 7.3 API Integration Pattern

Single lib/apiClient.js wraps every call:

```
apiClient.get(url, { params }).then(unwrapData) // returns response.data.data, or throws ApiError
```

`ApiError` carries `{ code, message, details }` straight from the backend's error envelope, so every feature's error UI can just render `error.message` or map `error.details` onto form fields — no per-feature parsing of the envelope.

8.4 7.4 Routing & Role Gating

- `ProtectedRoute` checks `isAuthenticated` (valid `accessToken` in the Zustand store).
- `RoleRoute` checks `user.role` against an `allowedRoles` prop and redirects to a 403 page otherwise. Every route declares its allowed roles right next to its path in `router.jsx`, so the whole app's access map is readable in one file.
- This is a UX convenience only — real enforcement happens in backend middleware (Section 8).

8.5 7.5 Design System

- **Colors:** neutral slate scale for chrome/backgrounds; a single indigo/blue primary for actions; semantic colors reserved for status only — green (approved/paid/active), amber (pending/draft), red (rejected/overdue), slate (closed/cancelled). No decorative gradients.
- **Typography:** one font family (Inter), 4 sizes for body/label/heading/display, one consistent line-height scale.
- **Spacing:** 4px base unit, Tailwind's default scale, no arbitrary pixel values.
- **Shared components built once, reused everywhere:** `<StatusBadge status="..." />` maps every status enum in the system to a consistent color + label; `<Table>` handles loading-skeleton rows, empty state, and pagination footer identically across Vendors/RFQs/Quotations/POs/Invoices lists.
- **Every list/detail page composes the same three states:** `<Skeleton />` while loading, `<EmptyState />` when the array is empty, `<ErrorState onRetry={refetch} />` on a query error — implemented once in `components/ui`, not re-implemented per feature.
- Once the `.html` design references are provided (Section 0), the exact palette/spacing/typography values here should be reconciled against them and this section updated to match the approved design precisely.

9 8. Authentication & Authorization

- **Password storage:** `bcryptjs`, `salt rounds = 12`.
- **Tokens:** short-lived JWT access token (15 min) carrying `{ userId, role }`, signed with `JWT_ACCESS_SECRET`; longer-lived refresh token (7 days), signed with `JWT_REFRESH_SECRET`, stored as an **httpOnly, secure, sameSite=strict cookie**. The access token is kept in memory on the frontend (Zustand store, not `localStorage`) to reduce XSS exposure; the `/auth/refresh` endpoint rotates both tokens.
- **Signup:** Procurement Officer / Manager / Admin accounts are created by an Admin (via `POST /users`) or seeded. **Public self-signup is exposed for the Vendor role only** — a vendor registers via `POST /auth/signup`, creating an account + Vendor record with status: "PENDING"; an Admin then activates it via `PATCH /vendors/:id/status`. This mirrors real procurement onboarding and gives the demo a clean "Admin approves a new vendor" beat.
- **Every protected route** runs `auth.middleware.js` (valid token required), then `role.middleware.js` (`requireRole([...])`) where relevant. Ownership checks happen inside the service layer where the role alone isn't enough — e.g. a Vendor can only submit a quotation against an RFQ they were actually assigned to, checked against the RFQ's `assignedVendors` array, not just their role claim.
- **No endpoint trusts the JWT role beyond identifying who is asking.** Every authorization decision that depends on a relationship (assigned vendor, RFQ owner, designated approver) is re-derived server-side from the database, not inferred from the token.
- **CORS:** locked to the deployed frontend origin only, `credentials: true` to allow the refresh cookie.
- **Rate limiting:** stricter limits on `POST /auth/login` and `POST /auth/forgot-password` to blunt brute-force/enumeration attempts.
- **Input validation:** Joi validates every request body/query/params before it reaches a controller — this doubles as injection protection (nothing malformed reaches Mongoose) alongside Mongoose's own parameterized query building.
- **Secrets:** `MONGODB_URI`, `JWT_ACCESS_SECRET`, `JWT_REFRESH_SECRET`, SMTP credentials all live in the backend's `.env` file, never committed (`.env` in `.gitignore`, `.env.example` committed with placeholder values), never sent to the frontend bundle.

10 9. MVP vs Optional Features

10.1 MVP (must work end-to-end for the demo)

- Auth: login, signup (vendor self-signup + seeded staff accounts), role-based routing.
- Dashboard with real widgets (not placeholders) per role.
- Vendor management: list, create, view, status, category, search/filter.
- RFQ: create (item details + deadline + attachment), assign vendors, publish, detail view.
- Quotation: vendor submit + edit-before-lock, officer views all quotations per RFQ.
- Comparison screen with lowest-price highlight and select action.
- Approval: request auto-created on selection, manager approve/reject with remarks.
- PO: auto-generated on approval, PO number, PDF/view.
- Invoice: generated from PO, tax + total calculation, PDF download, print, **email send** (real SMTP call, even to a test inbox).
- Activity log + notifications tied to every transition above.
- Basic analytics: spend total, RFQ count by status, top vendors by award count — real charts, not mockups.

10.2 Optional / Nice-to-have (only after MVP is solid)

- Full email-based forgot-password flow (can stub as “reset link generated” in server logs for the demo).
- Vendor performance rating auto-computed from delivery-timeliness history (start with a manually-set rating field).
- Exportable CSV reports — a straightforward add-on once the Reports API exists.
- Multi-level/multi-approver approval chains — MVP is single-approver.
- File preview (not just download) for RFQ attachments.
- Admin fine-grained permission editing beyond the 4 fixed roles.
- Real-time notification delivery (WebSocket/Socket.IO) — MVP can poll or refresh-on-navigation, which is invisible in a live demo.

11 10. Implementation Phases

Phase	Scope	Depends on
0. Foundation	Repo scaffolding, Mongoose models + indexes, seed script skeleton, apiResponse/error.middleware, Tailwind design tokens + UI primitives	-
1. Auth	Signup/login/JWT/refresh, role middleware, protected routing on frontend	Phase 0
2. Dashboard shell + layout	Sidebar/nav, role-gated routes, dashboard widgets wired to (initially stubbed) endpoints	Phase 1
3. Vendor management	Full CRUD + list/filter/search, vendor self-signup -> admin activation	Phase 1
4. RFQ	Create/publish/detail, vendor assignment, attachments (Multer)	Phase 3 (needs vendors to assign)
5. Quotations	Vendor submission + edit, officer's "quotations for this RFQ" view	Phase 4
6. Comparison + Selection	Aggregation endpoint, comparison UI, select-winner action	Phase 5
7. Approvals	Auto-created on selection, manager queue + decide	Phase 6
8. Purchase Orders	Auto-generate on approval, PO detail/PDF (PDFKit)	Phase 7
9. Invoices	Generate from PO, tax/total calc, PDF, print, email (Nodemailer)	Phase 8
10. Notifications + Activity Log	Centralized service wired retroactively into every Phase 4-9 transition	Phases 4-9
11. Reports & Analytics	Mongo aggregation pipelines + charts, real dashboard numbers replace Phase 2 stubs	Phase 9
12. Polish	Loading/empty/error state audit, responsive pass, seed data realism pass, security review (rate limits, headers, CORS), reconcile UI against .html design references	All above

Each phase should end with the relevant slice of the demo flow (Section 11) working live end-to-end, not just built in isolation.

12 11. Hackathon Demo Strategy (5-10 min)

1. **(0:00-0:30)** Open on the Procurement Officer dashboard already showing live numbers — sets the “this is a real running system” tone immediately.
2. **(0:30-2:00)** Create an RFQ live (mostly pre-filled, don’t type everything), assign 2 seeded vendors, publish.
3. **(2:00-2:30)** Switch to a Vendor account (pre-logged-in second browser tab) — show the RFQ appearing, submit a quotation. Switch to a second vendor tab, submit a different quotation — this is what makes comparison meaningful.
4. **(2:30-3:30)** Back to Officer: open Comparison, point out the lowest-price highlight and delivery difference, select a winner.
5. **(3:30-4:00)** Switch to Manager tab: approval request already sitting in queue, approve with a remark.
6. **(4:00-4:30)** Back to Officer: PO auto-generated, one click to generate Invoice.
7. **(4:30-5:30)** Show Invoice preview -> download PDF -> (optionally) send email live, show it land in the test inbox.
8. **(5:30-6:00)** Jump to Activity Log/Notifications to show the full trail was captured automatically, then Reports to show the dashboard/analytics numbers moved because of the transaction just performed.
9. Close by resizing to a mobile viewport for one screen (dashboard or comparison) to prove the responsiveness claim without spending demo time on it.

Keep 2-3 browser profiles/tabs pre-authenticated as each role before the demo starts — role-switching mid-demo via logout/login wastes time and risks a glitch on stage.

13 12. Risks

Risk	Area	Mitigation
Scope creep across 10 screens dilutes the core pipeline	Scope	Section 9's MVP list is the hard boundary; optional features only after Phase 12
Frontend/backend payload drift	Integration	Section 5 is frozen; frontend <code>api.js/*Schema.js</code> files hand-mapped from it, not invented ad hoc
PDF generation eating disproportionate time	Technical	Use PDFKit with a simple fixed layout, not pixel-perfect design; budget as its own sub-task in Phase 9
Email sending failing live during demo (network/SMTP)	Demo	Use a reliable test SMTP (Ethereal/Mailtrap) verified the day before; fallback to showing the generated PDF if live send fails
Race conditions on sequential PO/Invoice numbers if judges click fast	Technical	Number generation via atomic <code>findOneAndUpdate</code> on a Counter collection (Section 6.3)
Role-based UI hiding mistaken for real security	Security	Section 8 makes explicit: every authorization decision is re-checked server-side, not just role-gated on the frontend
Seed data reading as fake ("Test Vendor", round numbers)	UX/Demo credibility	Seed script uses varied realistic Indian company names/GSTINs, non-round prices, staggered historical dates (Section 6.3 seed folder)
Comparison screen becoming unreadable with many vendors	UX	Cap realistic seed/demo scenario at 3-4 vendors per RFQ; table designed to degrade to horizontal scroll beyond that, not break layout
.html design files not yet available	Design fidelity	Frontend UI primitives (Section 7.5) are built to reasonable defaults now; Section 0 flags exactly how to reconcile once files are uploaded, so no rework of component structure is needed, only styling values
Two developers implementing the same module differently	Team/process	Module folder pattern (<code>routes/controller/service/schema</code>) in Section 6.1 standardizes structure so any teammate can predict file layout

14 Amendment Log

(Add dated entries here if the frozen contract in Section 5 needs to change during implementation, with a one-line reason.)

- **v2.0** — Switched stack to MERN (MongoDB, Express, React, Node), plain JavaScript throughout (no TypeScript). Revised data model from relational (Prisma/Postgres) to Mongoose/MongoDB documents. Revised frontend and backend folder structures accordingly. Added full request/response payload samples to every API endpoint. Added Section 0 covering .html design reference file usage.
- **v1.0** — Initial proposal (MERN-adjacent stack originally specified as Postgres/Prisma/TypeScript) — superseded by v2.0.